

Moduli

Moduli

Come organizzare il codice in moduli e importarli in Python.

Perché è utile usare i moduli

Immagina di dover scrivere un programma grande. Mettere tutto il codice in un unico file lungo migliaia di righe sarebbe un disastro — impossibile da leggere o da modificare.

I **moduli** sono il modo in cui Python ti permette di suddividere il codice in file separati, organizzati per argomento. Ogni file `.py` è già un modulo. E Python viene fornito con centinaia di moduli già pronti da usare — la cosiddetta **libreria standard**.

Importare un modulo con `import`

Per usare un modulo, devi "importarlo" all'inizio del tuo file:

```
print(math.pi)           # 3.141592653589793 – il pi greco
print(math.sqrt(16))      # 4.0 – radice quadrata
print(math.floor(3.7))    # 3 – arrotondamento verso il basso
print(math.ceil(3.2))     # 4 – arrotondamento verso l'alto
```

Quando scrivi `math.sqrt`, stai dicendo: "usa la funzione `sqrt` che si trova nel modulo `math`".

Importare solo quello che ti serve con `from ... import`

Se sai già quali funzioni ti servono, puoi importarle direttamente. Così non devi scrivere il nome del modulo ogni volta:

```
from math import pi, sqrt, floor # Importo solo queste tre cose

print(pi)          # 3.141592653589793
print(sqrt(25))    # 5.0
print(floor(4.9))  # 4
```

`from math import *` — sconsigliato

Si può importare tutto in una volta con `*`, ma è una cattiva abitudine:

```
from math import * # Importa TUTTO dal modulo math

print(sin(0))     # Funziona, ma...
```

Il problema: non si capisce più da dove vengono le funzioni. Se due moduli hanno una funzione con lo stesso nome, una sovrascrive l'altra senza avvertirti.

Rinominare con `as`

Puoi dare un soprannome a un modulo o a una funzione con la parola `as`. Utile se il nome è lungo o se vuoi evitare conflitti:

```
print(m.pi)          # 3.14159...

from math import sqrt as radice_quadrata # Rinomino la funzione
print(radice_quadrata(9)) # 3.0
```

Alcune librerie molto usate hanno soprannomi convenzionali che tutti usano:

Creare il tuo modulo

Un modulo è semplicemente un file `.py`. Crea un file chiamato `calcolatrice.py`:

```
# calcolatrice.py – il mio modulo personale

def somma(a, b):
    return a + b

def sottrai(a, b):
    return a - b

def moltiplica(a, b):
    return a * b

pi = 3.14159
```

Poi in un altro file nella **stessa cartella**:

```
# main.py – uso il mio modulo

print(calcolatrice.somma(3, 4)) # 7
print(calcolatrice.pi)         # 3.14159
```

Il blocco `if __name__ == "__main__":`

Quando Python importa un modulo, esegue tutto il codice al suo interno. Questo può causare problemi se hai del codice di test che non vuoi far girare durante l'importazione.

La soluzione è racchiudere il codice di test in questo blocco speciale:

```
# calcolatrice.py

def somma(a, b):
    return a + b

if __name__ == "__main__":
    # Questo viene eseguito SOLO se avvii calcolatrice.py direttamente
    # NON viene eseguito se qualcuno fa "import calcolatrice"
    print(somma(3, 4)) # Test
```

Regola semplice: `__name__ == "__main__"` è `True` solo per il file che hai avviato tu, non per quelli importati.

Moduli utili già inclusi in Python

Modulo	A cosa serve
<code>math</code>	Funzioni matematiche (radici, trigonometria, logaritmi)
<code>random</code>	Numeri casuali
<code>datetime</code>	Date e orari
<code>os</code>	Interazione con il sistema operativo (file, cartelle)
<code>json</code>	Leggere e scrivere file JSON
<code>re</code>	Espressioni regolari (ricerche avanzate nel testo)
<code>time</code>	Misurare il tempo, mettere in pausa il programma

Il modulo **random** — numeri casuali

```
print(random.random())           # Numero decimale casuale
tra 0.0 e 1.0
print(random.randint(1, 10))     # Numero intero casuale tra
1 e 10 (inclusi)
print(random.choice(["mela", "banana", "uva"])) # Sceglie un elemento a
caso dalla lista

lista = [1, 2, 3, 4, 5]
random.shuffle(lista)           # Mescola la lista (modifica la lista originale)
print(lista)                    # Ordine casuale, es: [3, 1, 5, 2, 4]
```

Il modulo **OS** — interazione con il sistema

```
print(os.getcwd())               # Mostra in quale cartella ti trovi ora
print(os.listdir("."))           # Elenca i file nella cartella corrente
print(os.path.exists("file.txt")) # True se il file esiste, False
altrimenti
```